In [5]:

```
# Import libraries
import pandas as pd  # Data handling
import numpy as np  # Numerical operations
import matplotlib.pyplot as plt  # Data visualization

from sklearn.model_selection import train_test_split  # Train-test split
from sklearn.preprocessing import StandardScaler, LabelEncoder  #
Preprocessing
from sklearn.linear_model import LogisticRegression  # Logistic regression

from sklearn.metrics import accuracy_score, confusion_matrix,
precision_score, recall_score  # Model evaluation

# Create model
model = LogisticRegression()
```

In [6]:

```
# Import the dataset
df = pd.read_csv("Downloads/Social_Network_Ads.csv")
```

In [7]:

```
# Initialize the dataframe
df.head()
```

Out[7]:

	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	Male	19	19000	0
1	15810944	Male	35	20000	0
2	15668575	Female	26	43000	0
3	15603246	Female	27	57000	0
4	15804002	Male	19	76000	0

Data Preprocessing

In [8]:

```
# Convert categorical columns to numerical using Label Encoding
label_encoder = LabelEncoder()
```

In [10]:

```
df["Gender"] = label_encoder.fit_transform(df["Gender"])
```

In [11]:

```
df.head()
```

Out[11]:

	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	1	19	19000	0
1	15810944	1	35	20000	0
2	15668575	0	26	43000	0
3	15603246	0	27	57000	0
4	15804002	1	19	76000	0

In [12]:
```
# Check for null values
df.isnull().sum()
```

Out[12]:
```
User ID            0
Gender             0
Age                0
EstimatedSalary    0
Purchased          0
dtype: int64
```

In [13]:
```
# Compute Covariance Matrix
df.cov()
```

Out[13]:

	User ID	Gender	Age	EstimatedSalary	Purchased
User ID	5.134915e+09	-905.617719	-541.682870	1.737143e+08	244.836284
Gender	-9.056177e+02	0.250526	-0.386917	-1.031404e+03	-0.010201
Age	-5.416829e+02	-0.386917	109.890702	5.548738e+04	3.131165
EstimatedSalary	1.737143e+08	-1031.403509	55487.380952	1.162603e+09	5924.367168
Purchased	2.448363e+02	-0.010201	3.131165	5.924367e+03	0.230269

In [14]:
```
# Divide dataset into independent (X) and dependent (Y) variables
X = df.drop(columns=["Purchased"])  # Assuming "Purchased" is the target
variable
Y = df["Purchased"]
```

In [15]:
```
# Split dataset into training and testing datasets
xtrain, xtest, ytrain, ytest = train_test_split(X, Y, test_size=0.2,
random_state=42)
```

In [24]:
```
# Feature Scaling
from sklearn import preprocessing
# Apply Min-Max Scaling only on numerical columns
scaler = preprocessing.MinMaxScaler()
features_scaled = scaler.fit_transform(df)

# Create a new DataFrame with scaled features and original labels
df_normalized = pd.DataFrame(features_scaled, columns = df.columns)
```

In [25]:
```
df_normalized
```

Out[25]:

	User ID	Gender	Age	EstimatedSalary	Purchased
0	0.232636	1.0	0.023810	0.029630	0.0
1	0.982732	1.0	0.404762	0.037037	0.0
2	0.409926	0.0	0.190476	0.207407	0.0
3	0.147083	0.0	0.214286	0.311111	0.0
4	0.954801	1.0	0.023810	0.451852	0.0
...	...	...	...	...	...
395	0.503623	0.0	0.666667	0.192593	1.0
396	0.560787	1.0	0.785714	0.059259	1.0
397	0.352477	0.0	0.761905	0.037037	1.0
398	0.757720	1.0	0.428571	0.133333	0.0
399	0.110048	0.0	0.738095	0.155556	1.0

400 rows × 5 columns

In [26]:

```
# Train the Model (Logistic Regression)
logreg = LogisticRegression()
logreg.fit(xtrain, ytrain)
```

Out[26]:

```
LogisticRegression()
```

In [27]:

```
# Make Predictions
y_pred_train = logreg.predict(xtrain)
y_pred_test = logreg.predict(xtest)
```

In [28]:

```
# Model Evaluation
train_acc = accuracy_score(ytrain, y_pred_train)
test_acc = accuracy_score(ytest, y_pred_test)

cm = confusion_matrix(ytest, y_pred_test)
precision = precision_score(ytest, y_pred_test)
recall = recall_score(ytest, y_pred_test)
```

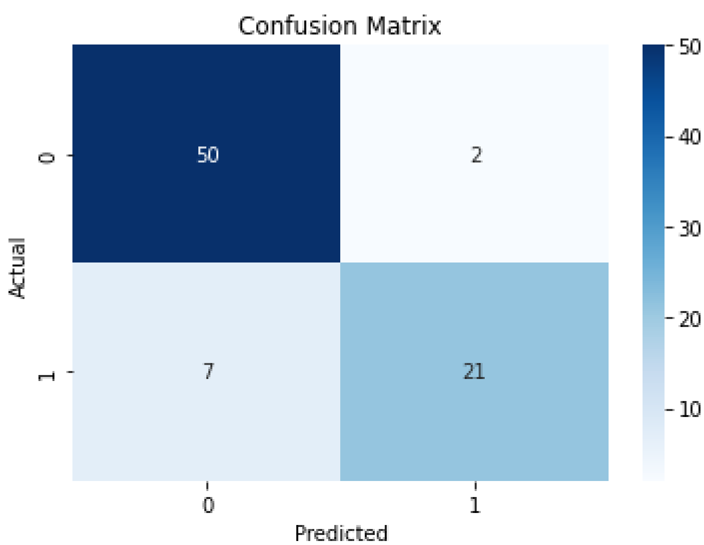
In [29]:

```
# Display results
print("Training Accuracy:", train_acc)
print("Testing Accuracy:", test_acc)
print("Confusion Matrix:\n", cm)
print("Precision:", precision)
print("Recall:", recall)
```

```
Training Accuracy: 0.821875
Testing Accuracy: 0.8875
Confusion Matrix:
 [[50  2]
 [ 7 21]]
Precision: 0.9130434782608695
Recall: 0.75
```

In [31]:

```
import seaborn as sns
# Visualizing Confusion Matrix
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```



In [ ]: